

APROJECT REPORT

On

Retrieval-Augmented Generation (RAG) Based Question Answering System

For the parliament fulfilling for the award of the degree of

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering

Submitted by

Priyanshi Sharma (202349624)

Vishal Dubey(2023000836)

Abhishek Kumar(2024464096)

Under the supervision of

Mr. Ayush Singh



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

SHARDA UNIVERSITY

GREATER NOIDA

Declaration

I hereby declare that the project titled “Retrieval-Augmented Generation (RAG) Based Intelligent Document Question Answering System” is an original work carried out by me under the supervision of my project guide. The work presented in this report has not been submitted previously for the award of any degree or diploma in any university or institution. All references used in this report have been properly acknowledged.

Signature:

Name: Priyanshi

Sharma System

ID: 2023492624

Signature:

Name: Vishal

Dubey System ID:

2023343293

Signature:

Name: Abhishek Kumar

Tiwari System ID:

2024464096

Acknowledgement

I would like to express my sincere gratitude to my project guide for their invaluable guidance, constructive criticism, and continuous encouragement throughout the development of this project. Their insights and technical expertise significantly enhanced the quality and direction of this work.

I am deeply thankful to the Head of the Department and faculty members of the Computer Science and Engineering Department for providing the academic foundation and resources necessary to complete this project successfully.

I would also like to extend my appreciation to my peers and friends for their support and discussions that contributed to refining my ideas and implementation strategies. Finally, I am grateful to my family for their constant encouragement and belief in my abilities.

Abstract

The rapid evolution of Artificial Intelligence and Natural Language Processing has significantly transformed the way machines process and generate human language. Transformer-based Large Language Models (LLMs) have demonstrated remarkable capabilities in text generation and comprehension. However, these models inherently suffer from a limitation: they rely solely on static pre-trained knowledge and cannot dynamically access external information during inference. This often leads to hallucinated or factually incorrect responses, especially in domain-specific applications.

To address this challenge, this project presents the design and implementation of a Retrieval-Augmented Generation (RAG) based intelligent document question answering system. The proposed system integrates semantic information retrieval with transformer-based generative modeling to produce context-aware and factually grounded responses. The architecture processes a domain-specific document, segments it into structured text chunks, converts them into dense vector embeddings using a pre-trained sentence transformer model, and stores them in a FAISS vector database for efficient similarity search.

Upon receiving a user query, the system retrieves the most semantically relevant document segments and supplies them as contextual input to a lightweight transformer model (FLAN-T5 Small) to generate the final response. The implementation is carried out entirely in Python and deployed using a Streamlit-based web interface for interactive usage.

Experimental evaluation indicates that the system significantly reduces hallucination, improves factual accuracy, and maintains efficient execution on standard hardware without requiring cloud-based APIs. The modular and scalable architecture ensures extensibility for multi-document and enterprise-level applications.

Keywords: Retrieval-Augmented Generation, Semantic Search, FAISS, Transformer Models, Natural Language Processing, Question Answering Systems.

Contents

Declaration	1
Acknowledgement	2
Abstract	3
1 Introduction	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Objectives	8
1.4 Scope of the Project	8
2 Literature Review	9
2.1 Traditional Information Retrieval Systems	9
2.2 Word Embedding Models	9
2.3 Transformer-Based Language Models	9
2.4 Retrieval-Augmented Generation.....	10
3 System Design and Architecture	11
3.1 Overall Architecture	11
3.2 Document Processing	11
3.3 Text Chunking Strategy.....	11
3.4 Embedding Generation	12
3.5 Vector Storage using FAISS.....	12
3.6 Query Processing and Retrieval	12
3.7 Response Generation.....	12
4 Hardware and Software Requirements	13
4.1 Introduction.....	13
4.2 Hardware Requirements.....	13
4.2.1 Minimum System Requirements.....	13
4.2.2 Recommended Configuration	14
4.3 Software Requirements	14

4.3.1	Operating System	14
4.3.2	Programming Language.....	14
4.3.3	Libraries Used	15
4.4	System Environment Setup.....	15
4.5	Advantages of the Setup.....	15
5	Research Methodology and Algorithm	16
5.1	Overview.....	16
5.2	Algorithmic Workflow.....	16
5.2.1	Step 1: Document Loading.....	16
5.2.2	Step 2: Text Preprocessing	16
5.2.3	Step 3: Text Chunking.....	16
5.2.4	Step 4: Embedding Generation	17
5.2.5	Step 5: FAISS Index Creation.....	17
5.2.6	Step 6: Query Processing.....	17
5.2.7	Step 7: Similarity Search	17
5.2.8	Step 8: Context Construction	17
5.2.9	Step 9: Response Generation.....	17
5.3	Pseudo Code	17
6	Implementation	19
6.1	Introduction.....	19
6.2	Document Loading and Preprocessing	19
6.3	Text Chunking Strategy.....	20
6.4	Embedding Generation	20
6.5	FAISS Index Construction	21
6.6	Query Processing and Retrieval	21
6.7	Transformer-Based Answer Generation.....	22
6.8	Web Interface Integration	22
7	Results and Performance Evaluation	23
7.1	Experimental Setup.....	23
7.2	Performance Metrics	23
7.3	Execution Time Analysis.....	24
7.4	Qualitative Evaluation.....	24
7.5	Discussion	24
8	Future Enhancements	25
8.1	Multi-Document Indexing.....	25
8.2	Persistent FAISS Storage	25

8.3	Conversational Memory.....	25
8.4	Hybrid Retrieval Strategy.....	25
8.5	Advanced Transformer Models.....	25
8.6	Cloud Deployment.....	25
8.7	Security Enhancements	26
9	Conclusion	27

Chapter 1

Introduction

1.1 Background

In recent years, Artificial Intelligence (AI) has become a transformative force across industries including healthcare, finance, education, and research. Among its various subfields, Natural Language Processing (NLP) has gained particular importance due to its ability to enable machines to understand, interpret, and generate human language.

Transformer-based architectures, introduced through the concept of self-attention mechanisms, revolutionized NLP by allowing models to capture contextual relationships within text more effectively than previous recurrent or convolutional approaches. Models such as BERT, GPT, and T5 demonstrated state-of-the-art performance in multiple language tasks.

However, despite their impressive capabilities, these models are fundamentally limited by the static nature of their training data. They cannot access updated knowledge or proprietary documents during inference. As a result, they may produce responses that are outdated, irrelevant, or fabricated.

1.2 Problem Statement

The primary problem addressed in this project is the inability of conventional Large Language Models to provide reliable, document-grounded answers for domain-specific queries without retraining.

When organizations attempt to use general-purpose LLMs for internal document search or academic assistance, they often encounter the following challenges:

- Generation of hallucinated or factually incorrect information.
- Inability to reference domain-specific documentation.
- High computational cost of fine-tuning large models.

- Dependence on external cloud APIs.

Therefore, there is a need for a system that integrates dynamic document retrieval with generative modeling while remaining lightweight and locally deployable.

1.3 Objectives

The objectives of this project are as follows:

- To design a Retrieval-Augmented Generation (RAG) based architecture.
- To implement semantic document indexing using vector embeddings.
- To integrate a transformer-based language model for context-aware response generation.
- To reduce hallucination in generated answers.
- To develop an interactive web-based interface for real-time question answering.

1.4 Scope of the Project

The scope of the project includes processing a domain-specific PDF document, generating embeddings, storing them in a FAISS vector database, and enabling interactive question answering. The system is designed for educational and small-scale enterprise applications and is fully executable on standard CPU-based machines.

Chapter 2

Literature Review

2.1 Traditional Information Retrieval Systems

Early question answering systems were based on keyword matching and rule-based logic. Classical Information Retrieval (IR) models such as TF-IDF and BM25 ranked documents using term frequency and inverse document frequency metrics.

While computationally efficient, these systems lacked semantic understanding and failed when queries used synonyms or paraphrased expressions.

2.2 Word Embedding Models

The introduction of distributed word representations such as Word2Vec and GloVe enabled semantic similarity detection. Words were represented as dense vectors in continuous space, allowing semantic relationships to be captured mathematically.

However, these embeddings were context-independent and could not handle polysemy effectively.

2.3 Transformer-Based Language Models

Transformers introduced contextual embeddings through self-attention mechanisms. Models like BERT and GPT significantly improved performance in reading comprehension and question answering tasks.

Despite these advancements, transformer models remained limited by static training knowledge and were prone to hallucination.

2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation integrates dense vector retrieval with generative models. Instead of relying solely on internal model parameters, the system retrieves relevant documents from an external knowledge base and uses them as conditioning context.

Research indicates that RAG architectures:

- Improve factual accuracy.
- Reduce hallucination.
- Enable domain-specific adaptability.
- Reduce the need for expensive fine-tuning.

However, most implementations depend on large cloud-hosted models, making them computationally expensive. This project focuses on a lightweight, locally deployable approach.

Chapter 3

System Design and Architecture

3.1 Overall Architecture

The proposed system follows a modular pipeline-based architecture consisting of the following components:

1. Document Processing Layer
2. Text Chunking Module
3. Embedding Generation Layer
4. Vector Database Layer (FAISS)
5. Query Processing Module
6. Similarity Retrieval Engine
7. Transformer-Based Generation Module
8. Web Interface Layer

3.2 Document Processing

The system begins by loading a PDF document and extracting its textual content using PyPDF. The extracted text undergoes preprocessing including whitespace normalization and removal of special characters.

3.3 Text Chunking Strategy

Large documents cannot be processed efficiently as a single input. Therefore, the text is segmented into overlapping chunks of 500 characters with 50-character overlap. This sliding-window approach ensures contextual continuity across segments.

Mathematically, if T is the total text length and C is chunk size, the number of chunks N is approximately:

$$N \approx \frac{T}{C - \text{Overlap}}$$

3.4 Embedding Generation

Each chunk is converted into a dense vector representation using the MiniLM-L6-v2 sentence transformer model. The embedding function can be expressed as:

$$E = f(\text{text})$$

where f is the transformer encoder mapping textual input into a high-dimensional vector space.

3.5 Vector Storage using FAISS

The embeddings are stored in a FAISS index structure, enabling efficient nearest-neighbor search. FAISS computes similarity using Euclidean distance or cosine similarity.

3.6 Query Processing and Retrieval

When a query Q is received, it is converted into an embedding E_Q . The similarity between E_Q and stored embeddings E_i is computed:

$$\text{Similarity}(E_Q, E_i) = \frac{E_Q \cdot E_i}{\|E_Q\| \|E_i\|}$$

Top-k most similar chunks are retrieved and concatenated to form contextual input.

3.7 Response Generation

The retrieved context is passed to the FLAN-T5 Small model along with the query. The model generates a response grounded in the provided context.

This layered architecture ensures modularity, scalability, and reduced hallucination.

Chapter 4

Hardware and Software Requirements

4.1 Introduction

The implementation of a Retrieval-Augmented Generation (RAG) based intelligent document question answering system requires a balanced combination of computational resources and software tools. Since the proposed architecture is designed for lightweight deployment, it avoids the need for high-end GPU infrastructure while maintaining efficient performance.

This chapter provides a detailed analysis of the hardware configuration, software dependencies, and execution environment used during development and testing.

4.2 Hardware Requirements

4.2.1 Minimum System Requirements

The system can operate efficiently on standard personal computers with the following configuration:

- Processor: Intel Core i3 (7th generation or above) or AMD equivalent
- RAM: 4 GB
- Storage: 5 GB free disk space
- Architecture: 64-bit operating system
- GPU: Not mandatory

These specifications are sufficient for:

- PDF processing
- Embedding generation (CPU-based inference)
- FAISS indexing
- Transformer-based response generation

4.2.2 Recommended Configuration

For smoother performance and reduced response latency, the following configuration is recommended:

- Processor: Intel Core i5/i7 or AMD Ryzen 5/7
- RAM: 8–16 GB
- Storage: SSD with at least 10 GB free space
- Optional GPU: CUDA-enabled NVIDIA GPU (for faster inference)

In experimental testing, the system demonstrated stable performance on an 8GB RAM machine with average response time below 3 seconds.

4.3 Software Requirements

4.3.1 Operating System

The system is cross-platform and compatible with:

- Windows 10/11
- Linux (Ubuntu 20.04+ recommended)
- macOS (Intel and Apple Silicon)

4.3.2 Programming Language

The entire implementation is developed using:

- Python 3.8 or above

Python was selected due to:

- Extensive NLP ecosystem

- Strong community support
- Availability of machine learning libraries
- Easy integration with web frameworks

4.3.3 Libraries Used

Library	Purpose
PyPDF	PDF text extraction
Sentence-Transformers	Semantic embedding generation
FAISS	Vector similarity search
Transformers	Language model integration
Torch	Deep learning backend
Streamlit	Web interface
NumPy	Numerical computations

Table 4.1: Software Libraries Used

4.4 System Environment Setup

Dependencies were installed using pip:

```
pip install streamlit sentence-transformers
pip install faiss-cpu transformers torch pypdf numpy
```

The application is executed using:

```
streamlit run app.py
```

The browser interface runs locally at:

<http://localhost:8501>

4.5 Advantages of the Setup

- Fully offline execution
- No reliance on external APIs
- Low computational overhead
- Suitable for academic and small enterprise usage

Chapter 5

Research Methodology and Algorithm

5.1 Overview

The methodology integrates document processing, semantic retrieval, and transformer-based text generation into a unified pipeline.

5.2 Algorithmic Workflow

5.2.1 Step 1: Document Loading

Input PDF file is loaded and text is extracted page-wise.

5.2.2 Step 2: Text Preprocessing

Preprocessing includes:

- Removal of extra whitespace
- Elimination of special characters
- Text normalization

5.2.3 Step 3: Text Chunking

Text is divided using sliding window mechanism.

Let:

$$ChunkSize = 500$$

$$Overlap = 50$$

5.2.4 Step 4: Embedding Generation

Each chunk is converted into vector:

$$E_i = f(chunk_i)$$

5.2.5 Step 5: FAISS Index Creation

Embeddings are added into vector index:

$$Index.add(E_i)$$

5.2.6 Step 6: Query Processing

User query Q is converted into embedding E_Q .

5.2.7 Step 7: Similarity Search

Compute similarity:

$$Similarity(E_Q, E_i)$$

Top-k vectors are retrieved.

5.2.8 Step 8: Context Construction

Retrieved chunks are concatenated.

5.2.9 Step 9: Response Generation

Input to transformer:

Context: Retrieved Text

Question: User Query

Output is generated response.

5.3 Pseudo Code

Load PDF

Extract Text

Split into Chunks

Generate Embeddings

Store in FAISS

While User Query:

Convert Query to Embedding

Retrieve Top-k Chunks

Form Context

Generate Answer

Display Output

Chapter 6

Implementation

6.1 Introduction

This chapter describes the complete practical implementation of the Retrieval-Augmented Generation (RAG) based document question answering system. The system is implemented using Python and structured in a modular fashion to ensure scalability, maintainability, and clarity.

The implementation consists of six major components:

1. Document Loading and Preprocessing
2. Text Chunking Mechanism
3. Embedding Generation
4. FAISS Index Construction
5. Query Processing and Retrieval
6. Transformer-Based Answer Generation

Each module is explained in detail below.

6.2 Document Loading and Preprocessing

The PDF document serves as the knowledge source. The document is loaded using the PyPDF library. Each page is processed sequentially, and textual content is extracted.

During preprocessing:

- Line breaks are normalized
- Extra whitespace is removed

- Special characters are cleaned
- Unicode inconsistencies are resolved

This preprocessing step ensures uniform formatting before semantic embedding.

6.3 Text Chunking Strategy

Since transformer models have input token limitations, processing entire documents at once is computationally inefficient.

Therefore, a sliding window approach is used:

- Chunk Size = 500 characters
- Overlap = 50 characters

Overlap ensures contextual continuity between adjacent segments.

If total document length is L , number of chunks N is approximately:

$$N = \frac{L}{\text{ChunkSize} - \text{Overlap}}$$

This segmentation improves retrieval precision and reduces memory consumption.

6.4 Embedding Generation

Each text chunk is converted into a dense semantic vector using:

Model: `sentence-transformers/all-MiniLM-L6-v2`

This model produces 384-dimensional embeddings.

Advantages:

- Lightweight architecture
- Fast CPU inference
- High semantic similarity accuracy
- Optimized for sentence-level encoding

Embedding transformation can be mathematically expressed as:

$$E_i = f(T_i)$$

Where:

- T_i = text chunk
- E_i = embedding vector
- f = transformer encoder function

6.5 FAISS Index Construction

The generated embeddings are stored using FAISS (Facebook AI Similarity Search).

Index type used:

IndexFlatL2

This index performs exact nearest-neighbor search using Euclidean distance:

$$D(E_Q, E_i) = \|E_Q - E_i\|^2$$

Where:

- E_Q = query embedding
- E_i = document embedding

FAISS allows efficient similarity search even for large embedding collections.

6.6 Query Processing and Retrieval

When the user submits a query:

1. Query is converted into embedding vector.
2. FAISS performs nearest neighbor search.
3. Top-k (k=3 or 5) most relevant chunks are retrieved.
4. Retrieved chunks are concatenated to form context.

Similarity is computed using cosine similarity:

$$\text{Similarity}(E_Q, E_i) = \frac{E_Q \cdot E_i}{\|E_Q\| \|E_i\|}$$

This ensures semantically relevant context selection.

6.7 Transformer-Based Answer Generation

Model Used:

google/flan-t5-small

FLAN-T5 is instruction-tuned and optimized for question answering tasks.

Input format:

Context: Retrieved Text

Question: User Query

The model generates a context-grounded response.

Advantages:

- Reduced hallucination
- Instruction-following capability
- Efficient inference
- CPU compatible

6.8 Web Interface Integration

The Streamlit framework provides a simple yet effective UI.

Features include:

- Text input box
- Real-time answer display
- Lightweight local deployment
- Easy integration

The modular implementation ensures that each component can be independently modified or upgraded.

Chapter 7

Results and Performance Evaluation

7.1 Experimental Setup

The system was tested under the following environment:

- Processor: Intel Core i5
- RAM: 8GB
- Operating System: Windows 11
- Python Version: 3.9

7.2 Performance Metrics

The system was evaluated based on:

- Retrieval Accuracy
- Response Relevance
- Hallucination Reduction
- Execution Time

7.3 Execution Time Analysis

Operation	Average Time
Document Indexing	15–25 seconds
Query Embedding	~ 1 second
FAISS Retrieval	~ 0.1 second
Response Generation	1–3 seconds

Table 7.1: Execution Time Analysis

7.4 Qualitative Evaluation

The system was tested using 25 domain-specific queries.

Observations:

- 92% responses were contextually accurate.
- Hallucination rate reduced significantly compared to standalone LLM.
- Retrieved chunks consistently matched query intent.
- Response clarity rated high during manual evaluation.

7.5 Discussion

The integration of semantic retrieval with generative modeling proved highly effective. Unlike traditional language models that rely solely on internal knowledge, the RAG architecture ensures grounding in document content.

The most significant improvement observed was reduction in fabricated information. Since the model is constrained by retrieved context, it generates more factual responses.

Chapter 8

Future Enhancements

Although the system performs effectively, several enhancements can improve scalability and robustness.

8.1 Multi-Document Indexing

Future versions can support multiple PDF documents with categorized indexing.

8.2 Persistent FAISS Storage

Saving and loading FAISS index from disk reduces startup time.

8.3 Conversational Memory

Adding memory buffer will enable multi-turn dialogue support.

8.4 Hybrid Retrieval Strategy

Combining keyword-based search (BM25) with semantic search can improve retrieval precision.

8.5 Advanced Transformer Models

Using larger models like FLAN-T5 Base or LLaMA can improve response fluency.

8.6 Cloud Deployment

Deploying on AWS or Azure enables multi-user access.

8.7 Security Enhancements

Adding authentication and encrypted document access for enterprise usage.

Chapter 9

Conclusion

This project successfully demonstrates the practical implementation of a Retrieval-Augmented Generation based intelligent document question answering system.

The proposed architecture effectively integrates semantic retrieval and transformer-based text generation to produce accurate and context-aware responses. By grounding generated answers in retrieved document content, the system significantly reduces hallucination and enhances reliability.

The lightweight design ensures that the system operates efficiently on CPU-based hardware without dependency on external APIs or large computational resources.

Experimental results confirm that retrieval-based grounding improves response accuracy compared to standalone language models. The modular architecture allows easy scalability and future enhancement.

In conclusion, this project highlights the importance of combining information retrieval with generative AI for building trustworthy and domain-specific intelligent systems. The implementation serves as a strong foundation for advanced knowledge management and AI-assisted documentation systems.